

Escalabilidade de programas limitados por memória e por CPU

Matheus Marinho

25 de agosto de 2026

1. Objetivo

O objetivo desta tarefa foi implementar e comparar dois programas paralelos em C com OpenMP. O primeiro realiza somas simples em vetores grandes e foi projetado para ser limitado pela movimentação de dados na memória. O segundo executa muitas operações aritméticas sobre cada elemento e foi projetado para ser limitado pela capacidade de cálculo da CPU.

Os dois programas foram medidos com diferentes números de threads para identificar três regiões: quando acrescentar threads melhora o desempenho, quando o ganho se estabiliza e quando a competição por recursos pode piorar o tempo. O Roofline Model foi usado para relacionar a intensidade aritmética de cada programa aos tetos sustentados de memória e de processamento.

2. Implementação

2.1 Programa limitado por memória

O núcleo do primeiro programa calcula, para vetores de N elementos:

$$c[i] = a[i] + b[i]$$

Cada iteração realiza apenas uma soma, mas precisa ler dois valores `double` e gravar um terceiro. Portanto, são pelo menos 24 bytes transferidos para uma única operação aritmética, ou aproximadamente 0,042 operação por byte. Essa baixa intensidade aritmética faz a largura de banda da memória se tornar o principal limite quando os vetores são maiores que a cache.

Foram usados 33.554.432 elementos por vetor. Os três vetores ocupam juntos 768 MiB, muito mais que os 24 MiB de cache compartilhada do processador testado. A coluna de GB/s representa tráfego útil mínimo: duas leituras e uma escrita. O tráfego físico pode ser maior por causa da política de escrita da cache.

2.2 Programa limitado por CPU

O segundo programa mantém quatro cadeias de recorrências aritméticas para cada elemento. Em cada uma das 200 iterações internas há várias multiplicações e somas. A carga completa realiza aproximadamente 3,2 bilhões de operações de ponto flutuante, enquanto os vetores de entrada e saída ocupam somente 15,3 MiB. Considerando 16 operações por iteração interna, uma leitura e uma escrita por elemento, sua intensidade aritmética útil é:

$$I_{CPU} = (16 \cdot 200) / 16 \text{ bytes} = 200 \text{ FLOP/byte}$$

Assim, cada valor carregado é reutilizado muitas vezes em registradores antes de um resultado ser escrito. A proporção entre cálculo e acesso à memória é muito maior que no primeiro programa, deslocando o gargalo para as unidades de execução da CPU.

2.3 Paralelização

Nos dois casos, as iterações do laço externo são independentes. A divisão do trabalho foi feita diretamente com:

```
#pragma omp parallel for schedule(static)
```

O escalonamento estático distribui blocos de iterações entre as threads e evita o custo de redistribuir trabalho durante a execução. O ajuste dinâmico de threads do OpenMP foi desativado com `omp_set_dynamic(0)`, e `omp_set_num_threads()` seleciona a quantidade usada em cada medição.

3. Método de medição

Cada configuração foi aquecida uma vez e medida cinco vezes. O valor apresentado é a mediana, menos sensível a interrupções ocasionais do sistema que a média. No teste de memória, cada amostra contém quatro somas completas e seu

tempo é dividido por quatro. O teste de CPU foi executado três vezes; sua tabela apresenta a mediana das três execuções completas, e cada uma delas já representa a mediana de cinco amostras.

Foi medido tempo de parede com `QueryPerformanceCounter` no Windows e `clock_gettime(CLOCK_MONOTONIC)` em sistemas POSIX. O código foi compilado com `gcc -O2 -Wall -Wextra -std=c99 -fopenmp`. A variável de controle e a leitura de amostras das saídas impedem que o compilador descarte os cálculos.

O computador de teste possui um Intel Core i7-13650HX, com 14 núcleos físicos (6 P-cores e 8 E-cores) e 20 processadores lógicos, 32 GiB de RAM e Windows 11. Não foi fixada afinidade: o posicionamento das threads foi decidido pelo escalonador do Windows. Portanto, os resultados representam o desempenho geral do programa nessa máquina, e não uma medição isolada de P-cores, E-cores ou SMT.

O *speedup* e a eficiência foram calculados por:

$$S(p) = T(1) / T(p) \quad \text{e} \quad E(p) = S(p) / p$$

4. Roofline Model

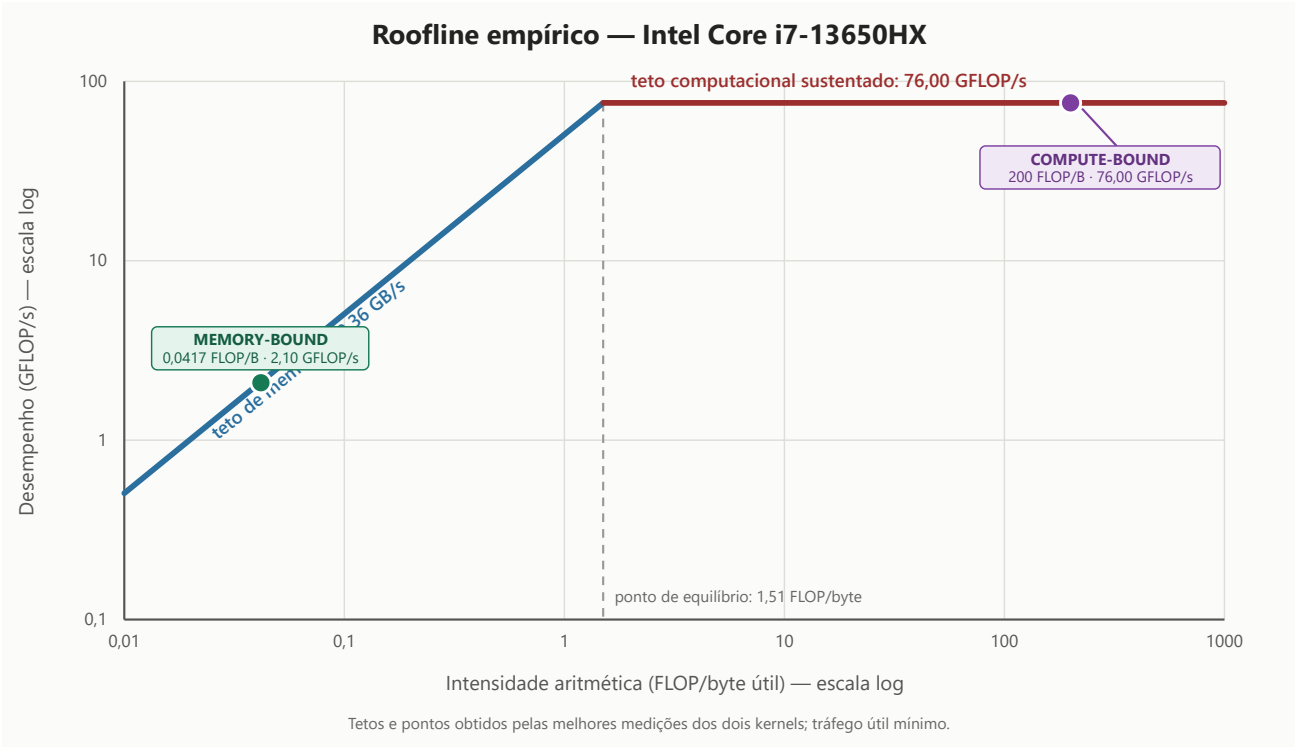
O Roofline Model limita o desempenho de um programa pelo menor entre o teto computacional e o produto da largura de banda pela intensidade aritmética:

$$P(I) = \min(P_{\text{máx}}, B_{\text{máx}} \cdot I)$$

Foi construído um Roofline empírico com os melhores valores sustentados neste experimento: $B_{\text{máx}} = 50,36 \text{ GB/s}$ na soma de vetores e $P_{\text{máx}} = 76,00 \text{ GFLOP/s}$ nas recorrências. O ponto de equilíbrio entre os dois tetos é:

$$I_{\text{equilíbrio}} = 76,00 / 50,36 = 1,51 \text{ FLOP/byte}$$

Programa	Intensidade	Desempenho observado	Limite do Roofline	Classificação
Soma de vetores	0,0417 FLOP/byte	2,10 GFLOP/s	2,10 GFLOP/s pela memória	Memory-bound
Recorrências	200 FLOP/byte	76,00 GFLOP/s	76,00 GFLOP/s pela CPU	Compute-bound



A soma de vetores está muito à esquerda do ponto de equilíbrio. A reta de memória prevê $50,36 \times 0,0417 = 2,10 \text{ GFLOP/s}$, exatamente a região observada. Acrescentar capacidade aritmética não resolve esse gargalo; seria necessário mover os dados mais rapidamente ou realizar mais trabalho com cada byte carregado.

As recorrências estão muito à direita do ponto de equilíbrio: a banda de memória permitiria uma taxa muito superior ao teto computacional, de modo que são as unidades de execução que limitam o programa. Os tetos foram obtidos dos próprios

kernels e representam desempenho sustentado, não o pico teórico de fábrica. Por isso, este gráfico é uma descrição empírica e não uma medição independente do pico absoluto da microarquitetura.

5. Resultados

5.1 Soma de vetores: *memory-bound*

Threads	Tempo (s)	Speedup	Eficiência	GB/s úteis	GFLOP/s
1	0,038428	1,000×	100,0%	20,96	0,873
2	0,027089	1,419×	70,9%	29,73	1,239
4	0,018967	2,026×	50,6%	42,46	1,769
6	0,018533	2,074×	34,6%	43,45	1,811
8	0,017474	2,199×	27,5%	46,08	1,920
10	0,017122	2,244×	22,4%	47,03	1,960
12	0,016805	2,287×	19,1%	47,92	1,997
14	0,016220	2,369×	16,9%	49,65	2,069
16	0,016240	2,366×	14,8%	49,59	2,066
20	0,015990	2,403×	12,0%	50,36	2,099

O desempenho melhorou rapidamente de uma para quatro threads: a largura de banda útil passou de 20,96 para 42,46 GB/s. A partir de quatro, o ganho marginal caiu porque várias threads passaram a compartilhar os mesmos canais de memória. Entre 10 e 20 threads, a largura de banda permaneceu aproximadamente entre 47 e 50 GB/s, caracterizando estabilização. Houve uma pequena oscilação em 16 threads, mas 20 threads produziram o melhor tempo, somente 1,4% abaixo do tempo com 14.

5.2 Recorrências aritméticas: *compute-bound*

Threads	Tempo (s)	Speedup	Eficiência	Gerações/s	GFLOP/s
1	0,533217	1,000×	100,0%	0,38	6,00
2	0,274339	1,944×	97,2%	0,73	11,66
4	0,141456	3,769×	94,2%	1,41	22,62
6	0,099725	5,347×	89,1%	2,01	32,09
8	0,085463	6,239×	78,0%	2,34	37,44
10	0,072273	7,378×	73,8%	2,77	44,28
12	0,062852	8,484×	70,7%	3,18	50,91
14	0,055674	9,578×	68,4%	3,59	57,48
16	0,049861	10,694×	66,8%	4,01	64,18
20	0,042108	12,663×	63,3%	4,75	76,00

O teste de CPU apresentou escalabilidade quase linear no começo: com quatro threads chegou a 3,769×. O crescimento continuou com retornos decrescentes, mas sem regressão na mediana consolidada. O melhor resultado ocorreu com 20 threads: 0,042108 s, *speedup* de 12,663× e 76,00 GFLOP/s.

6. Multithreading de hardware e os dois gargalos

O *hardware multithreading* permite que duas threads lógicas compartilhem um mesmo núcleo físico. Ele não duplica as unidades de execução, a cache nem a largura de banda. Seu efeito depende de quais recursos estavam ociosos.

No programa limitado por memória, uma thread frequentemente espera a chegada de dados. Enquanto uma está parada por uma falta de cache, outra thread lógica pode emitir instruções e manter mais requisições de memória em andamento. Isso pode esconder parte da latência. Neste experimento, 20 threads foram 1,4% mais rápidas que 14, mas o ganho foi pequeno porque a largura de banda total já estava próxima do teto de 50 GB/s.

No programa limitado por CPU, threads lógicas que compartilham um núcleo também compartilham unidades aritméticas, registradores, filas internas e cache. Se uma thread já mantiver esses recursos ocupados, outra pode gerar competição e fazer

o ganho estabilizar ou até piorar. Por outro lado, dependências ou desvios podem deixar ciclos ociosos que outra thread consegue aproveitar.

Neste experimento, o tempo do kernel de CPU continuou diminuindo até 20 threads. Como não foi controlada a afinidade em uma CPU híbrida, não é possível atribuir essa melhora especificamente ao multithreading de hardware. A conclusão correta é apenas que a configuração completa OpenMP + Windows foi a mais rápida entre as quantidades testadas. O efeito isolado de SMT dependeria de outro experimento, que não é exigido nesta tarefa.

7. Conclusão

Adicionar threads ajudou ambos os programas no início, mas por motivos e até limites diferentes. A soma de vetores atingiu rapidamente o teto compartilhado da memória: depois de quatro threads, os ganhos foram pequenos e a taxa estabilizou perto de 50 GB/s. A carga aritmética aproveitou mais threads por mais tempo e chegou a um *speedup* de 12,663× com 20 threads no ambiente testado.

O Roofline Model tornou essa diferença explícita. A intensidade de 0,0417 FLOP/byte posicionou a soma de vetores sob o teto inclinado da memória, enquanto 200 FLOP/byte posicionaram as recorrências sob o teto horizontal de processamento.

O experimento mostra que a melhor quantidade de threads não deve ser escolhida apenas pelo número de processadores lógicos. É necessário identificar o gargalo e medir: uma thread de hardware adicional pode esconder espera de memória, mas pode também disputar recursos que já estão ocupados em uma carga intensiva de CPU.

8. Referência

WILLIAMS, Samuel; WATERMAN, Andrew; PATTERSON, David. *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Technical Report UCB/EECS-2008-134, University of California, Berkeley, 2008. [Publicação e PDF original](#).


```

83     const int quantidade = (int)(sizeof(candidatos) / sizeof(candidatos[0]));
84     int usados = 0;
85     int i;
86
87     for (i = 0; i < quantidade && usados < capacidade; i++) {
88         if (candidatos[i] <= maximo) {
89             lista[usados++] = candidatos[i];
90         }
91     }
92     if (usados < capacidade && lista[usados - 1] != maximo) {
93         lista[usados++] = maximo;
94     }
95     return usados;
96 }
97
98 int main(int argc, char **argv)
99 {
100     size_t n = 32ULL * 1024ULL * 1024ULL;
101     int repeticoes = 4;
102     int max_threads;
103     int lista_threads[20];
104     int quantidade_threads;
105     double *a;
106     double *b;
107     double *c;
108     double tempo_base = 0.0;
109     size_t i;
110     int teste;
111
112     if (argc > 3) {
113         fprintf(stderr, "Uso: %s [elementos] [repeticoes]\n", argv[0]);
114         return EXIT_FAILURE;
115     }
116     if (argc >= 2) {
117         long long valor = ler_positivo(argv[1], "elementos");
118         if ((unsigned long long)valor > SIZE_MAX / sizeof(double)) {
119             fprintf(stderr, "Vetor grande demais.\n");
120             return EXIT_FAILURE;
121         }
122         n = (size_t)valor;
123     }
124     if (argc == 3) {
125         long long valor = ler_positivo(argv[2], "repeticoes");
126         if (valor > INT_MAX) {
127             fprintf(stderr, "Numero de repeticoes grande demais.\n");
128             return EXIT_FAILURE;
129         }
130         repeticoes = (int)valor;
131     }
132
133     a = malloc(n * sizeof(*a));
134     b = malloc(n * sizeof(*b));
135     c = malloc(n * sizeof(*c));
136     if (a == NULL || b == NULL || c == NULL) {
137         fprintf(stderr, "Nao foi possivel alocar %.1f MiB.\n",
138             3.0 * n * sizeof(double) / (1024.0 * 1024.0));
139         free(a);
140         free(b);
141         free(c);
142         return EXIT_FAILURE;
143     }
144
145     for (i = 0; i < n; i++) {
146         a[i] = 1.0 + (double)(i % 97) * 0.001;
147         b[i] = 2.0 - (double)(i % 89) * 0.001;
148     }
149
150     omp_set_dynamic(0);
151     max_threads = omp_get_num_procs();
152     quantidade_threads = montar_lista_threads(max_threads, lista_threads, 20);
153
154     printf("Benchmark memory-bound: c[i] = a[i] + b[i]\n");
155     printf("Elementos por vetor: %zu | memoria total: %.1f MiB\n", n,
156         3.0 * n * sizeof(double) / (1024.0 * 1024.0));
157     printf("Mediana de %d amostras, %d repeticoes por amostra\n",
158         AMOSTRAS, repeticoes);
159     printf("Intensidade aritmetica: %.6f FLOP/byte util\n\n", 1.0 / 24.0);
160     printf("threads,tempo_s,speedup,eficiencia_pct,largura_GB_s,GFLOP_s\n");
161
162     for (teste = 0; teste < quantidade_threads; teste++) {
163         int threads = lista_threads[teste];
164         double tempos[AMOSTRAS];
165         double mediana;
166         double checksum = 0.0;
167         double bytes;
168         int amostra;
169         int r;
170
171         omp_set_num_threads(threads);

```

```

172     somar_vetores(a, b, c, n); /* aquecimento e primeira alocação física */
173
174     for (amostra = 0; amostra < AMOSTRAS; amostra++) {
175         double inicio = walltime();
176         for (r = 0; r < repeticoes; r++) {
177             somar_vetores(a, b, c, n);
178         }
179         tempos[amostra] = (walltime() - inicio) / repeticoes;
180     }
181
182     qsort(tempos, AMOSTRAS, sizeof(tempos[0]), comparar_double);
183     mediana = tempos[AMOSTRAS / 2];
184     if (teste == 0) {
185         tempo_base = mediana;
186     }
187     for (i = 0; i < n; i += 4096) {
188         checksum += c[i];
189     }
190     controle += checksum;
191
192     /* Trafego util minimo: duas leituras e uma escrita por elemento. */
193     bytes = 3.0 * n * sizeof(double);
194     printf("%d,%.9f,%.3f,%.1f,%.2f,%.3f\n", threads, mediana,
195           tempo_base / mediana,
196           100.0 * tempo_base / (mediana * threads),
197           bytes / mediana / 1.0e9,
198           (double)n / mediana / 1.0e9);
199 }
200
201 if (controle == 0.123456789) {
202     printf("controle: %.9f\n", controle);
203 }
204 free(a);
205 free(b);
206 free(c);
207 return EXIT_SUCCESS;
208 }

```

10. Código-fonte: programa limitado por CPU

cpu_bound.c · 211 linhas

```
1  /*
2  * Tarefa 4 - Benchmark limitado pela capacidade de calculo da CPU
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c99 -fopenmp cpu_bound.c -o cpu_bound.exe -lm
6  *
7  * Executar (argumentos opcionais: elementos e iteracoes internas):
8  * ./cpu_bound.exe
9  * ./cpu_bound.exe 1000000 200
10 */
11
12 #define _POSIX_C_SOURCE 199309L
13
14 #include <errno.h>
15 #include <limits.h>
16 #include <omp.h>
17 #include <stdio.h>
18 #include <stdlib.h>
19 #include <stdint.h>
20
21 #ifdef _WIN32
22 #include <windows.h>
23 #else
24 #include <time.h>
25 #endif
26
27 #define AMOSTRAS 5
28
29 static volatile double controle = 0.0;
30
31 static double walltime(void)
32 {
33 #ifdef _WIN32
34     LARGE_INTEGER contador;
35     LARGE_INTEGER frequencia;
36     QueryPerformanceCounter(&contador);
37     QueryPerformanceFrequency(&frequencia);
38     return (double)contador.QuadPart / (double)frequencia.QuadPart;
39 #else
40     struct timespec tempo;
41     clock_gettime(CLOCK_MONOTONIC, &tempo);
42     return tempo.tv_sec + tempo.tv_nsec / 1000000000.0;
43 #endif
44 }
45
46 static int comparar_double(const void *a, const void *b)
47 {
48     double x = *(const double *)a;
49     double y = *(const double *)b;
50     return (x > y) - (x < y);
51 }
52
53 static long long ler_positivo(const char *texto, const char *nome)
54 {
55     char *fim;
56     long long valor;
57
58     errno = 0;
59     valor = strtoll(texto, &fim, 10);
60     if (errno != 0 || *texto == '\0' || *fim != '\0' || valor <= 0) {
61         fprintf(stderr, "Valor invalido para %s: %s\n", nome, texto);
62         exit(EXIT_FAILURE);
63     }
64     return valor;
65 }
66
67 /*
68 * Quatro cadeias de dependencias independentes mantem as unidades aritmeticas
69 * ocupadas. Ha muitas operacoes para apenas uma leitura e uma escrita.
70 */
71 static void calcular(const double *entrada, double *saida, size_t n,
72                     int iteracoes)
73 {
74     long long i;
75
76     #pragma omp parallel for schedule(static)
77     for (i = 0; i < (long long)n; i++) {
78         double x1 = entrada[i] + 0.101;
79         double x2 = entrada[i] + 0.202;
80         double x3 = entrada[i] + 0.303;
81         double x4 = entrada[i] + 0.404;
82         int k;
```

```

83
84     for (k = 0; k < iteracoes; k++) {
85         x1 = x1 * 0.99999991 + x2 * 0.00000017 + 0.00000011;
86         x2 = x2 * 0.99999989 + x3 * 0.00000019 + 0.00000013;
87         x3 = x3 * 0.99999987 + x4 * 0.00000023 + 0.00000017;
88         x4 = x4 * 0.99999983 + x1 * 0.00000029 + 0.00000019;
89     }
90     saida[i] = x1 + x2 + x3 + x4;
91 }
92 }
93
94 static int montar_lista_threads(int maximo, int *lista, int capacidade)
95 {
96     const int candidatos[] = {1, 2, 4, 6, 8, 10, 12, 14, 16, 20,
97                               24, 32, 48, 64, 96, 128};
98     const int quantidade = (int)(sizeof(candidatos) / sizeof(candidatos[0]));
99     int usados = 0;
100     int i;
101
102     for (i = 0; i < quantidade && usados < capacidade; i++) {
103         if (candidatos[i] <= maximo) {
104             lista[usados++] = candidatos[i];
105         }
106     }
107     if (usados < capacidade && lista[usados - 1] != maximo) {
108         lista[usados++] = maximo;
109     }
110     return usados;
111 }
112
113 int main(int argc, char **argv)
114 {
115     size_t n = 1000000;
116     int iteracoes = 200;
117     int max_threads;
118     int lista_threads[20];
119     int quantidade_threads;
120     double *entrada;
121     double *saida;
122     double tempo_base = 0.0;
123     size_t i;
124     int teste;
125
126     if (argc > 3) {
127         fprintf(stderr, "Uso: %s [elementos] [iteracoes]\n", argv[0]);
128         return EXIT_FAILURE;
129     }
130     if (argc >= 2) {
131         long long valor = ler_positivo(argv[1], "elementos");
132         if ((unsigned long long)valor > SIZE_MAX / sizeof(double)) {
133             fprintf(stderr, "Vetor grande demais.\n");
134             return EXIT_FAILURE;
135         }
136         n = (size_t)valor;
137     }
138     if (argc == 3) {
139         long long valor = ler_positivo(argv[2], "iteracoes");
140         if (valor > INT_MAX) {
141             fprintf(stderr, "Numero de iteracoes grande demais.\n");
142             return EXIT_FAILURE;
143         }
144         iteracoes = (int)valor;
145     }
146
147     entrada = malloc(n * sizeof(*entrada));
148     saida = malloc(n * sizeof(*saida));
149     if (entrada == NULL || saida == NULL) {
150         fprintf(stderr, "Nao foi possivel alocar %.1f MiB.\n",
151             2.0 * n * sizeof(double) / (1024.0 * 1024.0));
152         free(entrada);
153         free(saida);
154         return EXIT_FAILURE;
155     }
156     for (i = 0; i < n; i++) {
157         entrada[i] = 0.5 + (double)(i % 1000) * 0.0001;
158     }
159
160     omp_set_dynamic(0);
161     max_threads = omp_get_num_procs();
162     quantidade_threads = montar_lista_threads(max_threads, lista_threads, 20);
163
164     printf("Benchmark compute-bound: recorrencias aritmeticas independentes\n");
165     printf("Elementos: %zu | iteracoes internas: %d | dados: %.1f MiB\n", n,
166         iteracoes, 2.0 * n * sizeof(double) / (1024.0 * 1024.0));
167     printf("Mediana de %d amostras\n", AMOSTRAS);
168     printf("Intensidade aritmetica: %.1f FLOP/byte util\n\n",
169         (double)iteracoes);
170     printf("threads,tempo_s,speedup,eficiencia_pct,Giteracoes_s,GFLOP_s\n");
171

```

```

172     for (teste = 0; teste < quantidade_threads; teste++) {
173         int threads = lista_threads[teste];
174         double tempos[AMOSTRAS];
175         double mediana;
176         double checksum = 0.0;
177         int amostra;
178
179         omp_set_num_threads(threads);
180         calcular(entrada, saida, n, iteracoes); /* aquecimento */
181
182         for (amostra = 0; amostra < AMOSTRAS; amostra++) {
183             double inicio = walltime();
184             calcular(entrada, saida, n, iteracoes);
185             tempos[amostra] = walltime() - inicio;
186         }
187
188         qsort(tempos, AMOSTRAS, sizeof(tempos[0]), comparar_double);
189         mediana = tempos[AMOSTRAS / 2];
190         if (teste == 0) {
191             tempo_base = mediana;
192         }
193         for (i = 0; i < n; i += 1024) {
194             checksum += saida[i];
195         }
196         controle += checksum;
197
198         printf("%d,%.9f,%.3f,%.1f,%.2f,%.2f\n", threads, mediana,
199             tempo_base / mediana,
200             100.0 * tempo_base / (mediana * threads),
201             (double)n * iteracoes / mediana / 1.0e9,
202             16.0 * n * iteracoes / mediana / 1.0e9);
203     }
204
205     if (controle == 0.123456789) {
206         printf("controle: %.9f\n", controle);
207     }
208     free(entrada);
209     free(saida);
210     return EXIT_SUCCESS;
211 }

```